

Security Audit

Orion Finance (DeFi)

Table of Contents

Executive Summary	3
Project Context	3
Audit Scope	5
Security Rating	7
Intended Program Functions	8
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	42
Conclusion	43
Our Methodology	44
Disclaimers	46
About Hashlock	47

Executive Summary

Orion Finance partnered with Hashlock to conduct a security audit of their code. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Orion Finance is a decentralized infrastructure layer for onchain portfolio management, enabling asset managers to launch and operate ERC-4626 vaults with verifiable execution and optional strategy privacy. Users queue deposits and redemptions, while strategists submit target portfolio intents. At each epoch, Orion commits the relevant protocol state onchain and uses a zkVM to compute the next portfolio state and aggregate trade requirements. The resulting proof is verified onchain before execution and settlement, ensuring that portfolio updates follow the intended strategy without requiring the strategy itself to be publicly exposed.

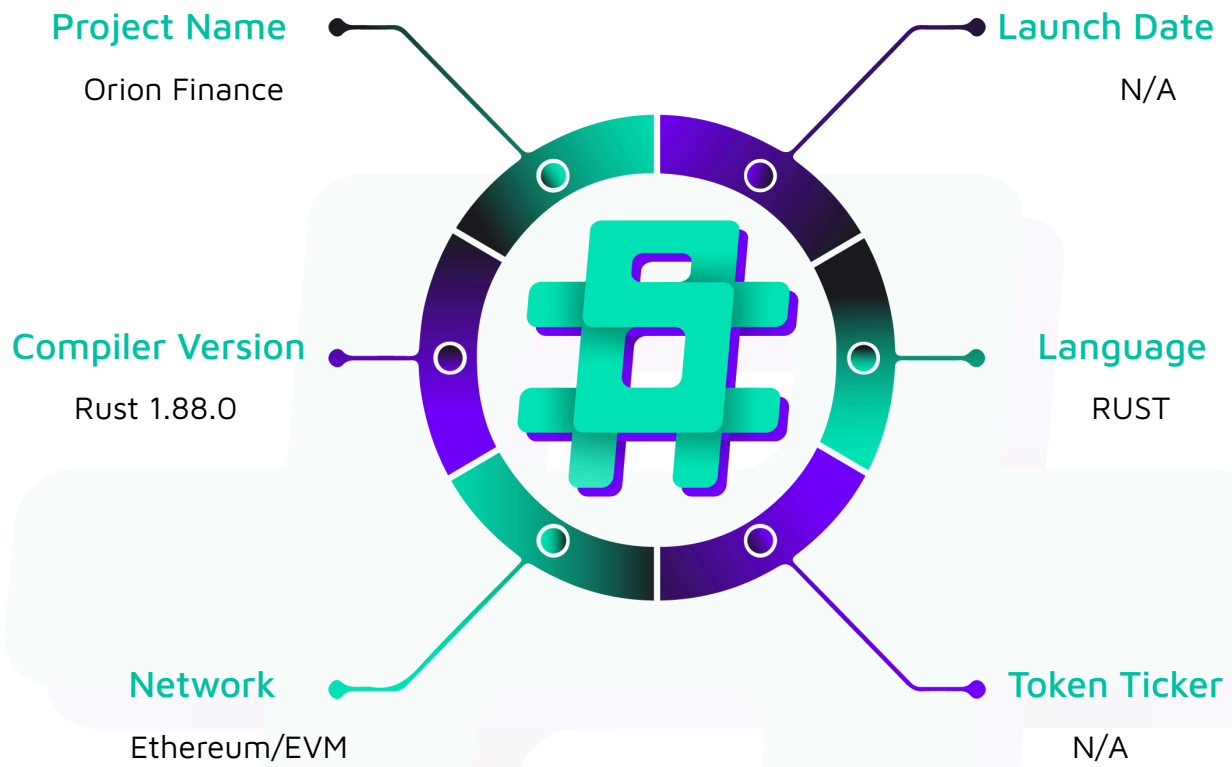
Project Name: Orion Finance

Project Type: DeFi / zero-knowledge state-transition and proving infrastructure

Website: <https://www.orionfinance.ai/>

Logo:



Visualised Context:

Audit Scope

We at Hashlock audited the Rust code within the Orion Finance, the scope of work included a comprehensive review of the programs and modules listed below. We tested the programs to assess their security, functionality, and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Orion Finance zk-Orchestrator
Platform	Rust/SP1 zkVM
Audit Date	August, 2026
Program 1	lib/src/lib.rs
Program 2	lib/src/vault_fees.rs
Program 3	lib/src/emit.rs
Program 4	lib/src/confidential.rs
Program 5	lib/src/hpke.rs
Program 6	lib/src/utils.rs
Program 7	lib/src/crypto.rs
Program 8	lib/src/sol.rs
Program 9	lib/src/types.rs
Program 10	program/src/main.rs
Program 11	script/src/bin/main.rs
Audited GitHub Commit Hash	dc8af5960b3933d6af3a433c07c79dabdf358f7d
Fix Review GitHub Commit Hash	760959d9bfdc86ecbf832cb5e0d9b3879d841168

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.



Security Rating

After Hashlock's Audit, we found the codebase to be **"Secure"**. The code all follows simple logic, with correct and detailed ordering. They use a series of interfaces and external dependencies, including standard open-source libraries.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Program Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

4 High severity vulnerabilities

4 Medium severity vulnerabilities

3 Low severity vulnerabilities

3 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Program Functions

Claimed Behaviour	Actual Behaviour
lib/src/lib.rs - Serves as the primary deterministic epoch state-transition engine. - Processes vault deposits and redemption requests. - Values initial vault portfolios and calculates their resulting total assets.	Program achieves this functionality.
lib/src/emit.rs - Converts portfolio weights into target token shares. - Converts asset balances into underlying-denominated values and converts target values back into token amounts.	Program achieves this functionality.
lib/src/vault_fees.rs - Converts shares to assets and assets to shares using point-in-time vault totals and the vault's virtual-share offset - Calculates annual management fees. - Calculates performance fees for the supported absolute, hurdle-rate, and high-water-mark models.	Program achieves this functionality.
lib/src/hpke.rs - Implements the low-level cryptographic routines used for confidential vault data. - Implements HPKE Base mode using X25519, HKDF-SHA256, and AES-128-GCM. models.	Program achieves this functionality.

lib/src/crypto.rs - Builds the protocol-state commitment that binds guest configuration and state to the public input used for proving. - Defines the final per-vault state representation.	Program achieves this functionality.
lib/src/sol.rs - Defines Rust representations of the Solidity structures exchanged with the on-chain Liquidity Orchestrator. - Calculates the committed leaf for each vault from its state, portfolio, intent, requests, and fee configuration.	Program achieves this functionality.
lib/src/types.rs - Defines the serialisable witness structures supplied to the SP1 guest. - Represents per-vault deposits, redemptions, portfolios, intents, total supply, total assets, and encrypted data.	Program achieves this functionality.
lib/src/utils.rs - Converts integer values between token decimal domains. - Applies floor rounding when reducing decimal precision.	Program achieves this functionality.
program/src/main.rs - Serves as the SP1 zkVM guest entrypoint. - Reads a serialised GuestEpochInput from SP1 standard input. - Executes the deterministic epoch transition through the guest library.	Program achieves this functionality.
script/src/bin/main.rs	Program achieves this

- | | |
|---|-----------------------|
| <ul style="list-style-type: none">- Serves as the host-side prover and fixture-generation application.- Reads protocol configuration, epoch state, vault state, token balances, prices, fee models, and confidential data from the configured blockchain RPC endpoint. | functionality. |
|---|-----------------------|

Code Quality

This audit scope involves the programs of Orion Finance, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Orion Finance program code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] `lib/src/crypto.rs#build_protocol_state_hash` - Prover-controlled derived parameters are not bound to the committed protocol state

Description

`GuestEpochInput` accepts `intent_factor`, `price_adapter_precision`, `underlying_asset`, and `underlying_decimals` as independent witness fields. Their canonical values can be derived from configuration already represented in the committed protocol state, but the guest neither derived these values nor bound the independently supplied copies to `build_protocol_state_hash()`.

Vulnerability Details

The guest input contains both the committed source configuration and separate derived parameters:

```
pub struct GuestEpochInput {  
    // ...  
    pub epoch_duration: u32,  
    pub intent_factor: U256,  
    pub price_adapter_precision: U256,  
    pub underlying_asset: Address,  
    pub underlying_decimals: u8,  
    pub price_adapter_decimals: u8,  
    pub strategist_intent_decimals: u8,  
    // ...  
}
```

```
intent_factor == U256::from(10u64).pow(U256::from(strategist_intent_decimals));  
price_adapter_precision == U256::from(10u64).pow(U256::from(price_adapter_decimals));  
underlying_asset == assets[0];
```

```
underlying_decimals == token_decimals[0];
```

The reference prover constructs these values correctly, but a trustless prover can bypass the reference API and serialize `GuestEpochInput` directly. In the current implementation, `build_protocol_state_hash()` committed the decimal configuration, ordered asset universe, and token decimals, but did not commit the four separately supplied derived values or verify the relationships above.

Impact

A malicious trustless prover can generate a valid proof for a transition that does not correspond to the committed protocol configuration. This can corrupt trades, valuations, fees, deposits, redemptions, vault portfolios, and protocol buffer accounting, and can introduce secondary cryptographic risk for encrypted outputs.

Recommendation

Remove the redundant private fields and derive them inside the guest from committed configuration. Otherwise, include them in the protocol-state hash and assert every relationship explicitly, including the canonical underlying-asset address and all decimal-to-precision conversions. The output-encryption context should additionally bind a unique transition or output commitment, and the integration layer should retain one accepted output commitment throughout an epoch.

Status

Resolved

[H-02] lib/src/lib.rs#build_output_commit_payload - Uncommitted trailing deposits create phantom vault or buffer assets

Description

The commitment reads one `pending_deposits[i]` value for each entry in `state.vaults`, but output construction later sums the entire `pending_deposits` vector. No guest assertion requires the vector length to equal `vaults.len()`.

Vulnerability Details

The vault commitment reads only `pending_deposits[i]` for each real vault:

```
compute_vault_leaf(
    *vault,
    state.vault_is_encrypted(i),
    state.vault_is_decommissioning(i),
    &state.vault_fee_models[i],
    vd.pending_redeems[i].iter().sum(),
    vd.pending_deposits[i],
    vd.total_supplies[i],
    vd.total_assets[i],
    portfolio_hash,
    intent_hash,
)
```

Later, `build_output_commit_payload()` sums the entire vector:

```
let deposits_processed: U256 = state
    .vault_state_data
    .pending_deposits
    .iter()
    .copied()
    .sum();
```

A prover can append an extra value `X` after the committed vault entries. The input commitment stays unchanged, but `deposits_processed` increases by `X`. The guest can then assign that unbacked amount to vault holdings or the protocol buffer even though no tokens were deposited.

Impact

A valid proof can make total vault and buffer accounting exceed the orchestrator's real ERC-20 balance. Subsequent trades or withdrawals may fail, or real pooled assets belonging to other vaults and depositors may be consumed to satisfy fictitious accounting claims.

Recommendation

Require exact lengths for every vault- and asset-parallel vector before either commitment or computation. Prefer one fixed per-vault structure instead of independent vectors, and add an in-guest conservation assertion proving that all final vault assets, fees, redemption outflows, and buffer changes are backed by committed opening assets and real committed inflows.

Status

Resolved

[H-03] lib/src/vault_fees.rs#performance_fee_benchmark - Fee type IDs are decoded with a different enum order

Description

The Rust guest assigns different meanings to fee type values 1, 2, and 3 than the companion Solidity vault enum, while `script/src/bin/main.rs` copies the numeric `feeType` into `GuestFeeModel` without translation.

Vulnerability Details

The onchain fee model crosses the ABI boundary as an untyped numeric `feeType`:

```
struct FeeModel {
    uint8 feeType;
    uint16 performanceFee;
    uint16 managementFee;
    uint256 highWaterMark;
}
```

The Rust guest assigns the following meanings to those numbers:

```
match fee_type {
    0 => current_share_price, // Absolute
    1 => high_water_mark,    // HighWaterMark
    3 => get_hurdle_price(
        current_share_price,
        risk_free_rate,
        epoch_duration,
    ),                        // HardHurdle
    4 => high_water_mark.max(get_hurdle_price(
        current_share_price,
        risk_free_rate,
        epoch_duration,
    )),                      // HurdleHwm
    2 => unreachable!("SoftHurdle does not use this function."),
    _ => panic!("Invalid fee type: {fee_type}"),
}
```

The companion Solidity ordering is:

```
0 = Absolute  
1 = Soft Hurdle  
2 = Hard Hurdle  
3 = High Water Mark  
4 = Hurdle + High Water Mark
```

Values 1, 2, and 3 therefore select different formulas in Solidity and Rust. The numeric value is still committed correctly, so the proof succeeds while applying the wrong fee model. Before publication, the report should cite the exact pinned companion enum declaration.

Impact

Vault users can be systematically overcharged or undercharged, and fee benchmark state can advance according to a model that the manager and depositors did not select. The error can propagate into later epochs and other accounting derived from post-fee total assets.

Recommendation

Generate the Solidity and Rust enum definitions from one shared source, or use explicit cross-language numeric constants with checked decoding. Add an integration test that passes every numeric fee type through the companion contract, host witness construction, and guest calculation and verifies that all layers select the same model.

Status

Resolved

[H-04] lib/src/vault_fees.rs#performance_fee_amount - Performance fees are incorrectly annualized

Description

The guest scales crystallized profit by `epoch_duration / SECONDS_PER_YEAR` before applying the configured performance-fee percentage. This time factor belongs to recurring management fees, not to a percentage charged on realized gains.

Vulnerability Details

The performance-fee branch first scales eligible profit by `epoch_duration / SECONDS_PER_YEAR` and only then applies the fee rate:

```
mul_div(
    mul_div(
        profits,
        U256::from(epoch_duration),
        U256::from(SECONDS_PER_YEAR),
    ),
    U256::from(performance_fee),
    U256::from(BASIS_POINTS_FACTOR),
)
```

That time factor is appropriate for the recurring management-fee calculation:

```
let management_fee = mul_div(
    mul_div(
        U256::from(fee_model.management_fee),
        active_total_assets,
        U256::from(BASIS_POINTS_FACTOR),
    ),
    U256::from(epoch_duration),
    U256::from(SECONDS_PER_YEAR),
);
```

It is not appropriate for a performance fee charged directly on realized profit. With a one-day epoch, a configured 20% fee becomes roughly `20% / 365`, so a profit of 1,000,000 units produces about 548 units instead of 200,000. The missing fee may not be recovered after the benchmark or high-water mark advances.

Impact

Managers and the protocol can lose almost the entire intended performance fee at each crystallization event. Post-fee vault accounting, redemption values, and subsequent benchmark behavior are also calculated from the systematically undercharged amount.

Recommendation

Apply the configured performance-fee basis points directly to eligible profit. Retain `epoch_duration / SECONDS_PER_YEAR` only in the management-fee branch, and add fixed-vector tests for daily, weekly, and annual epochs across every supported fee model.

Status

Resolved

Medium

[M-01] lib/src/crypto.rs#compute_epoch_state_commit_internal - Aggregate redemption commitment omits output-critical queue partitioning

Description

The vault leaf commits only the sum of pending redemption shares, while the guest converts each individual request to assets separately and floors every conversion before summing the results.

Vulnerability Details

The committed vault leaf includes only the sum of all pending redemption shares:

```
compute_vault_leaf(
    *vault,
    state.vault_is_encrypted(i),
    state.vault_is_decommissioning(i),
    &state.vault_fee_models[i],
    vd.pending_redeems[i].iter().sum(),
    vd.pending_deposits[i],
    vd.total_supplies[i],
    vd.total_assets[i],
    portfolio_hash,
    intent_hash,
)
```

The transition instead converts each individual request separately:

```
convert_to_assets_with_pit_total_assets(
    user_shares,
    after_fees,
    total_supply,
    state.underlying_decimals,
)
```

Because each conversion floors, `convert(a + b)` can differ from `convert(a) + convert(b)`. The host also discards the requester list and keeps only the share vector:

```
let (_users, shares) =
  <(Vec<Address>, Vec<U256>) as SolValue>::abi_decode_params(raw.as_ref())
    .expect("Failed to decode pendingRedeemBatch");
shares
```

A trustless prover can therefore choose a different request partition with the same committed aggregate and obtain different redemption outflows, liquidity reservations, or trade legs.

Impact

A trustless prover can produce different redemption payouts and settlement decisions for the same committed aggregate redemption amount. This can underpay a batch, defer processing, or change the trades and liquidity reservations used by the epoch.

Recommendation

Commit the ordered redemption batch, including each requester and share amount, and validate its exact length. Alternatively, convert the committed aggregate once and distribute the result according to a separately committed deterministic dust-allocation rule shared by the guest and vault contract.

Status

Resolved

[M-02] script/src/bin/main.rs#main - Unsolicited token transfers can make the reference prover reject a committed epoch

Description

The reference host reconstructs the epoch witness from token balances and then applies exact custody checks. Permissionless ERC-20 transfers can therefore make the host either reproduce a different input commitment or reject an otherwise harmless balance surplus.

Vulnerability Details

The reference prover has two donation-related failure paths. Unless a block is configured, it reads `Latest` instead of the block used for the frozen commitment:

```
fn fetch_block_id_from_env() -> BlockId {
    match std::env::var(FETCH_BLOCK_NUMBER_ENV) {
        // An explicitly configured block number is handled above.
        // ...
        _ => BlockNumberOrTag::Latest.into(),
    }
}
```

It then loads live token balances into the witness:

```
let mut lo_balances = Vec::with_capacity/assets.len());
for &token in &assets {
    lo_balances.push(call!(
        token,
        IERC20::balanceOfCall { account: lo },
        "LO bal"
    ));
}
let lo_balance_underlying = lo_balances[u_idx];
```

An underlying-token donation changes the reconstructed commitment. A non-underlying donation can instead fail the later exact-balance invariant:

```
if commit != epoch_state.epochStateCommitment {
    eprintln!(
```



```

        "[zk-orchestrator/error] Commit mismatch. Cannot continue with proof
generation."
    );
    std::process::exit(1);
}

if let Err(e) = assert_payload_invariants(&guest, &witness, &states) {
    eprintln!(
        "[zk-orchestrator/error] payload invariant failed before fixture dump: {e}"
    );
    std::process::exit(1);
}

```

That invariant rejects any surplus not assigned to a vault:

```

} else if lhs != rhs {
    return Err(Violation {
        kind: "loBalance",
        detail: format!(
            "asset[{i}] {}: expected  $\Sigma$ portfolio={rhs} actual L0={lhs}",
            input.assets[i]
        ),
    });
}

```

These are availability failures in the reference host, not proof-soundness failures in the SP1 guest. If this host is the required production path, a negligible direct transfer can repeatedly delay proving until operators use the correct historical block or reconcile the surplus.

Impact

A permissionless token transfer can repeatedly prevent the reference production pipeline from generating or publishing a proof for an active epoch. The issue causes operational denial of service and recovery overhead, but it does not by itself make an invalid SP1 proof acceptable.

Recommendation

Record the exact commitment block hash and query every witness value at that EIP-1898 block. Store or expose frozen balance snapshots where practical. For current custody checks, model unsolicited surplus explicitly, accept `actual >= accounted`, and provide a deterministic reconciliation or sweep path instead of treating every positive difference as fatal.

Status

Resolved

[M-03] lib/src/vault_fees.rs#get_hurdle_price - Epoch hurdle return is rounded before price multiplication

Description

The guest first converts the annual hurdle into an integer number of epoch basis points and floors that intermediate result. It then applies the already-rounded rate to the share price.

Vulnerability Details

`get_hurdle_price()` floors the annual rate to whole epoch basis points before applying it to `current_share_price`:

```
let hurdle_return = mul_div(
    U256::from(risk_free_rate),
    U256::from(epoch_duration),
    U256::from(SECONDS_PER_YEAR),
);

mul_div(
    current_share_price,
    U256::from(BASIS_POINTS_FACTOR)
        .checked_add(hurdle_return)
        .expect("overflow"),
    U256::from(BASIS_POINTS_FACTOR),
)
```

For a 5% annual hurdle and daily epochs, the exact rate is about 1.36986 basis points, but the guest applies only 1. Shorter epochs can round the hurdle to zero. Hurdle-based performance fees can therefore trigger against a lower threshold than configured.

Impact

Users of hurdle-based vaults can be charged performance fees on returns that should remain below the configured hurdle, with the discrepancy recurring at every short epoch.

Recommendation

Apply the share price, annual rate, and epoch duration in one full-precision expression and divide only once, for example `price * (BPS * YEAR + rate * duration) / (BPS * YEAR)`. Preserve the intended rounding direction with a reviewed `mul_div` implementation.

Status

Resolved

[M-04] lib/src/lib.rs#build_output_commit_payload - Encrypted vault portfolios are emitted in plaintext

Description

For every encrypted vault, `build_output_commit_payload()` places the final tokens and shares arrays in `VaultState` while also sealing the same portfolio into `portfolioCiphertext`. The complete `StatesStruct` is then ABI-encoded as public settlement data.

Vulnerability Details

`VaultState` stores the plaintext tokens and shares together with `portfolioCiphertext`. The same output type is used for both transparent and encrypted vaults:

```
struct VaultState {
    bool processRedeem;
    uint256 totalAssetsForRedeem;
    uint256 totalAssetsForDeposit;
    uint256 finalTotalAssets;
    uint256 managementFee;
    uint256 performanceFee;
    address[] tokens;
    uint256[] shares;
    bytes portfolioCiphertext;
}
```

For an encrypted vault, `build_output_commit_payload()` fills all three fields from the same final portfolio:

```
VaultState {
    processRedeem: process_redeem_per_vault[vault_idx],
    totalAssetsForRedeem: vaults_total_assets_for_fulfill_redeem[vault_idx],
    totalAssetsForDeposit: vaults_total_assets_for_fulfill_deposit[vault_idx],
    finalTotalAssets: vaults_total_assets[vault_idx],
    managementFee: vaults_management_fee[vault_idx],
    performanceFee: vaults_performance_fee[vault_idx],
    tokens: vaults_final_tokens[vault_idx].clone(),
    shares: vaults_final_shares[vault_idx].clone(),
}
```

```
portfolioCiphertext: portfolio_ciphertexts[vault_idx].clone(),  
}
```

The complete `StatesStruct` is ABI-encoded into public `statesBytes`. Anyone can therefore read the plaintext portfolio directly, regardless of whether the receiving contract later ignores those arrays. Encrypting the same data in parallel does not provide confidentiality.

Impact

Every observer can recover the complete portfolio of each encrypted vault. This defeats the intended privacy boundary, exposes manager strategies, and can facilitate strategy copying and trade anticipation.

Recommendation

Use separate output representations for transparent and encrypted vaults. An encrypted-vault output should contain only `portfolioCiphertext` and deliberately public accounting fields. Plaintext token and share arrays must remain private inside the guest and must not be included in `calldata`, fixtures, events, or output structures.

Status

Resolved

Low

[L-01] Dockerfile, .github/workflows/ci.yml, script/setup-vk-map.sh, and script/pull-elf.sh - Build inputs and proving artifacts are not consistently authenticated

Description

Several setup and build paths execute remote installers or select downloaded artifacts without enforcing a separately controlled checksum, signature, immutable reference, or expected ELF-to-vKey mapping.

Vulnerability Details

Some CI dependencies use mutable release tags instead of immutable commits:

```
- uses: actions/checkout@v4
- uses: bnjbvr/cargo-machete@v0.9.2
- uses: actions/setup-go@v5
- uses: dtolnay/rust-toolchain@stable
- uses: actions/cache@v4
```

script/setup-vk-map.sh downloads and extracts archives without checking an approved digest:

```
curl -fSL --retry 8 --retry-delay 5 -C - -o "${stage}/${tarball}" "$url"
tar -xzf "${stage}/${tarball}" -C "$stage"
rm -f "${stage}/${tarball}"
```

script/pull-elf.sh selects the latest successful artifact from main rather than an ELF tied to the audited commit and an independently approved hash:

```
RUN_ID=$(gh run list --repo "$REPO" --workflow "$WORKFLOW" --branch main
--status success --limit 1 --json databaseId --jq '.[0].databaseId')
```

```
gh run download "$RUN_ID" --repo "$REPO" --name "$ARTIFACT_NAME" --dir  
"$TMPDIR"
```

A compromised installer, action tag, CI account, or artifact store could therefore execute substituted build code or provide a different guest ELF. The repository has some pinned paths, but the protection is not applied consistently.

Impact

A compromised upstream or CI boundary can execute attacker-controlled build code or replace the proving program used by operators, potentially creating an ELF and vKey that do not correspond to the audited source.

Recommendation

Pin every action and tool to an immutable commit or digest, verify all downloaded archives and installers with approved checksums or signatures, and maintain a separately controlled mapping from audited commit to expected ELF SHA-256 and deployed SP1 vKey.

Status

Resolved

[L-02] lib/src/lib.rs#is_empty_vault - Incomplete empty-vault checks can clear non-empty state

Description

`is_empty_vault()` checks only that a vault's pending deposit and cached total assets are zero. It ignores total supply, pending redemptions, plaintext or encrypted portfolio holdings, and other request state.

Vulnerability Details

`is_empty_vault()` checks only `pending_deposits` and cached `total_assets`:

```
pub(crate) fn is_empty_vault(
    state: &GuestEpochInput,
    vault_idx: usize,
) -> bool {
    state.vault_state_data.pending_deposits[vault_idx] == U256::ZERO
        && state.vault_state_data.total_assets[vault_idx] == U256::ZERO
}
```

When both are zero, the guest skips normal portfolio processing:

```
for vault_idx in 0..n_vaults {
    if is_empty_vault(state, vault_idx) {
        initial_vault_portfolios.push(vec![U256::ZERO; n_assets]);
        continue;
    }
    // normal portfolio processing
}
```

The predicate ignores total supply, pending redemptions, plaintext holdings, and encrypted state. If the committed accounting is already inconsistent, the guest can emit an empty portfolio instead of rejecting the state. The need for a pre-existing inconsistency is why this remains low severity.

Impact

If cached total assets diverge from the actual vault state, holdings or user requests can be omitted from the proven output. The prerequisite inconsistent state lowers the likelihood and severity.

Recommendation

Treat a vault as empty only when total supply, every request queue, all plaintext or encrypted portfolio holdings, and total assets are consistently zero. Any disagreement between these independent indicators should abort the guest.

Status

Resolved

[L-03] Makefile#vkey - Documented vKey workflows reference a missing script

Description

Makefile, script/setup.sh, script/pull-elf.sh, script/sync-elf.sh, and script/diagnose-vkey.sh invoke script/run-vkey.sh, but that script is absent from the audited repository.

Vulnerability Details

The documented vKey target invokes script/run-vkey.sh:

```
vkey:
    @./script/run-vkey.sh
```

That file is absent, even though several setup, synchronization, and diagnostic scripts also reference it. The official ELF-to-vKey verification flow therefore fails and may push operators toward ad hoc or skipped checks.

Impact

Operators cannot reliably execute the documented ELF-to-vKey validation workflow, increasing the chance of deployment mistakes and inconsistent proving or verifier configuration.

Recommendation

Restore script/run-vkey.sh with a reviewed and pinned implementation, or replace every caller with one canonical maintained command. Add CI coverage that executes all documented vKey setup and diagnostic paths.

Status

Resolved

QA

[Q-01] script/entrypoint-aws.sh - ECS publication rejects the fixture filename produced by the prover

Description

The fixture writer creates zk-orchestrator_groth16-fixture_<epoch>_<commitment-prefix>.json, while script/entrypoint-aws.sh parses only zk-orchestrator_groth16-fixture_<epoch>.json.

Vulnerability Details

The fixture writer always adds a commitment prefix to the filename:

```
let path = dir.join(format!(
    "zk-orchestrator_groth16-fixture_{}_{}.json",
    epoch_counter,
    hex::encode(&input_commitment.as_slice()[..8])
));
```

The ECS entrypoint accepts only a filename ending directly after the epoch number:

```
filename=$(basename "${latest_fixture}")
if [[ ${filename} =~ zk-orchestrator_groth16-fixture_([0-9]+)\.json ]]; then
    epoch="${BASH_REMATCH[1]}"
else
    echo "[prover] Could not parse epoch from fixture filename: ${filename}" >&2
    exit 1
fi
```

The regex therefore rejects every current fixture after proving and before S3 publication. Because script/entrypoint-aws.sh is outside the enumerated Rust scope, this is an operational QA issue.

Impact

The supplied ECS/Fargate handoff can fail to publish a valid generated proof through its intended artifact path.

Recommendation

Have the prover return the exact generated path in machine-readable form and upload that file directly. Otherwise, update the parser to validate both the epoch and commitment suffix, remove stale fixtures before each run, and include the full commitment in the S3 key.

Status

Resolved

[Q-02] `contracts/LiquidityOrchestrator.sol#performUpkeep` - The integration does not document or enforce the 255-window settlement boundary

Description

The Rust guest can emit arbitrarily many sell, buy, and vault-operation entries, while the companion orchestrator tracks minibatch sizes and progress with `uint8`. With a batch size of one, the 256th window cannot be represented.

Vulnerability Details

The Rust output uses dynamic arrays and does not encode a maximum number of settlement windows:

```
struct SellLegOrders {
    address[] sellingTokens;
    uint256[] sellingAmounts;
    uint256[] sellingEstimatedUnderlyingAmounts;
}

struct BuyLegOrders {
    address[] buyingTokens;
    uint256[] buyingAmounts;
    uint256[] buyingEstimatedUnderlyingAmounts;
}
```

The companion integration is reported to use `uint8` minibatch cursors, so the 256th window cannot be represented. Current limits may make this unreachable, and the exact Solidity source is outside the supplied scope, so the report keeps it as a QA integration boundary that should be pinned and documented.

Impact

The undocumented boundary can become an epoch-liveness problem if future configuration permits more than 255 settlement windows; otherwise it remains a missing cross-repository invariant and boundary test.

Recommendation

Document and enforce a common maximum before an epoch begins, or widen the companion cursor and batch-size types. Add integration tests for 255 and 256 windows and ensure oversized outputs are rejected before any partial settlement.

Status

Resolved

[Q-03] lib/src/confidential.rs#expand_intent_slot - Malformed non-empty encrypted intents fail open to a hold transition

Description

When a non-empty encrypted intent fails HPKE authentication, canonical decoding, or semantic validation, `expand_intent_slot()` clears the intent arrays and returns `true`, causing the guest to retain the existing portfolio rather than reject the witness.

Vulnerability Details

For a non-empty encrypted intent, both decryption failure and semantic-validation failure clear the intent and return `true`:

```
let Ok((mut toks, mut weights)) = decrypt_intent(sk, intent_ct) else {
    vd.intent_tokens[v] = Vec::new();
    vd.intent_weights[v] = Vec::new();
    return true;
};

if validate_decrypted_intent(
    &mut toks,
    &mut weights,
    intent_factor,
    u,
).is_err() {
    vd.intent_tokens[v] = Vec::new();
    vd.intent_weights[v] = Vec::new();
    return true;
}
```

The returned flag is later interpreted as a hold transition:

```
if state.vault_ignore_intent(vault_idx) {
    let held = hold_open_to_planning_tv1(open, planning_tv1, state);
    let (sell_q, buy_q) = bootstrap_trade_queues(open, &held, u);
    return (held.clone(), held, sell_q, buy_q);
}
```

A corrupt, wrong-key, or malformed intent therefore produces a successful proof that keeps the existing portfolio instead of failing. This hides the difference between “no

intent” and “invalid intent” and conflicts with the documented fail-closed behavior. Direct attacker reachability depends on who can provide the committed ciphertext.

Impact

An invalid encrypted intent can silently leave a vault in its previous allocation for the epoch instead of halting and surfacing the failure. This can prevent an intended risk reduction or rebalance and can leave users exposed to an allocation the manager believed would be changed.

Recommendation

Reserve raw empty bytes for the explicit unset state. Any non-empty ciphertext that fails authentication, canonical decoding, or semantic validation should abort the guest. If hold-on-error is intentionally retained, update the specification and commit an explicit failure status that downstream settlement and monitoring must acknowledge before continuing.

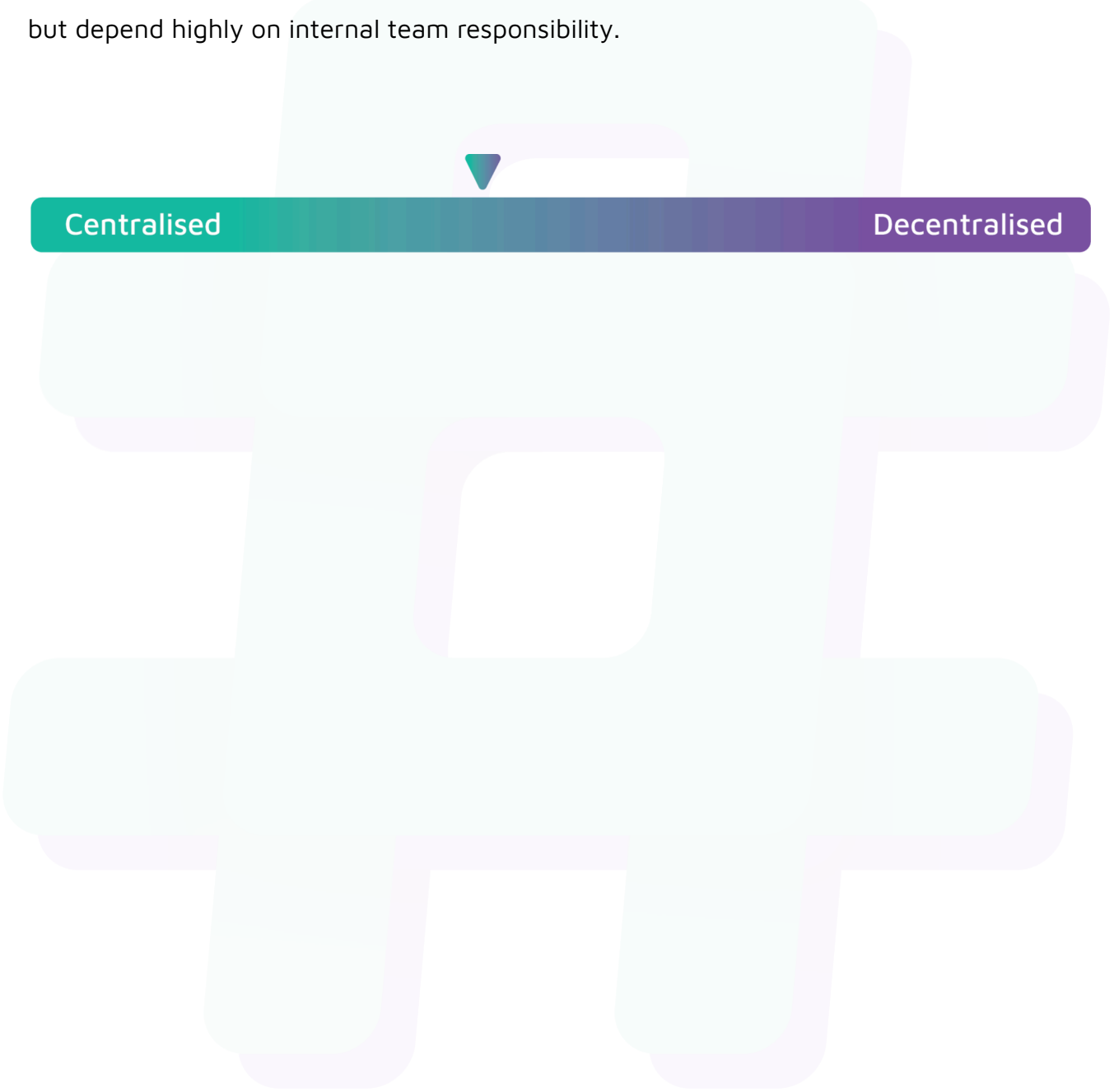
Status

Resolved

Centralisation

The Orion Finance project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Orion Finance project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.